Pii

System Administrator's Manual

Pii Project

Version 0.1.0

pi-improved

Preface

Pii is a single static Rust binary. It is a terminal-native AI coding agent, a personal assistant daemon, a reactive IoT/automation engine, and a self-improving cognitive layer, all in one process. This manual tells a system administrator everything needed to install it, configure it, run it, and debug it.

The companion *Technical Manual* covers the architecture and the technical analysis for developers. The two manuals complement each other: this one answers *how to operate*, the other answers *how it works*. Cross-references are marked inline.

Contents

Contents iii

1 Overview 1
- 1.1 The binary . . . 1
- 1.2 The daemon . . . 1
- 1.3 The workspace . . . 1

2 Installation 2
- 2.1 Prerequisites . . . 2
- 2.2 Build from source . . . 2
- 2.3 Install the binary . . . 2
- 2.4 Optional companion binaries . . . 2

3 Configuration 3
- 3.1 The config file . . . 3
- 3.2 The `[core]` section . . . 3
- 3.3 The `[net]` section . . . 3
- 3.4 The `[[worker]]` section . . . 3
- 3.5 The `[[channel]]` section . . . 4
- 3.6 The `[memory]` section . . . 4
- 3.7 The `[[cron]]` section . . . 4
- 3.8 The `[evolve]` section . . . 4
- 3.9 Environment variables . . . 4
- 3.10 Runtime directory resolution . . . 5

4 Running Pii 6
- 4.1 One-shot prompt (`pii -p`) . . . 6
- 4.2 The daemon (`pii daemon`) . . . 6
- 4.3 The TUI (`pii tui`) . . . 6
- 4.4 Graphs . . . 7
- 4.5 Control commands . . . 7
- 4.6 Soak (stress) testing . . . 8

5 The Web Viewer 9

6 The MCP Bridge 10

7 The Self-Improving Boss 11

8 Debugging 12
- 8.1 Check the daemon is up . . . 12
- 8.2 No model configured . . . 12
- 8.3 No usable runtime dir . . . 12
- 8.4 Capability denied . . . 12
- 8.5 Graph lint errors . . . 13
- 8.6 Tracing a run . . . 13
- 8.7 Approvals . . . 13
- 8.8 Heal ledger . . . 13
- 8.9 Logs . . . 14

9 Operational Notes **15**
9.1 Single-writer store 15
9.2 Feature-gated subsystems 15
9.3 Where to go for more 15

Overview 1

Pii ("Pi Improved") is a single static Rust binary that unifies four capabilities:

1. **Coding agent core** — a terminal-native AI coding agent with sub-100ms cold start, structured concurrency, capability-gated context, JSONL sessions, and bash mediation.
2. **Assistant/gateway** — a personal assistant daemon with channels, LLM providers, tools, and an SOP engine.
3. **Cognitive memory + model tiering** — entities and relationships, tier-aware tool selection.
4. **Self-improvement loop** — skill extraction, procedural templates, trajectory logging.

The full vision and the four pillars are documented in the SSOT at `_-docs/01-overview/001-pii-vision.md`. The architecture is in the companion Technical Manual, Chapter 1.

1.1 The binary

The primary binary is `pii` (from the `pii-cli` crate). It is a single static binary; there is no runtime install surface. Version `0.1.0`.

1.2 The daemon

Pii runs as a daemon (`pii daemon`) that owns the store and the control socket. The daemon is the single writer to the redb store; all control flows over a local-only Unix socket (mode 0600). See Chapter 4 for how to run it.

1.3 The workspace

The project is a Rust workspace of crates. The kernel ABI is `pii-abi` (traits + types only, no transport deps), enforced by `xtask deps-check` and a hard file-size gate. The crates are described in the Technical Manual, Chapter 1.

Installation 2

2.1 Prerequisites

- A Linux host (the daemon uses Unix sockets and peer-credential authorization).
- Rust toolchain to build from source (the project is a Cargo workspace). See `rust-toolchain.toml` for the pinned toolchain.
- A model provider. Pii is model-agnostic; the default provider chain is documented in the project `AGENTS.md`. At minimum you need one configured model (see Chapter 3).

2.2 Build from source

Build

```
cd /projects/pi-improved
cargo build --release
# the binary lands at target/release/pii
```

The workspace builds a monolith minus feature-gated subsystems. To include an optional subsystem (e.g. the TUI or the self-improving boss), build with its feature:

Build with features

```
cargo build --release --features tui,evolve
```

The feature-gating convention is documented in the SSOT at `_docs/02-guides/feature-gating.md`. The default build is headless (`pii -p` and `pii daemon` work without the TUI).

2.3 Install the binary

Copy the built binary to a location on `PATH`:

Install

```
cp target/release/pii /usr/local/bin/pii
pii --version   # prints "pii 0.1.0"
```

2.4 Optional companion binaries

- `pii-web` — a read-only web viewer (see Chapter 5).
- `pii-mcp` — an MCP server + client (see Chapter 6).
- `pii-evolve` — the self-improving boss launcher (see Chapter 7).

Configuration 3

3.1 The config file

Pii reads a single TOML config file. The path is resolved in this order:

1. `$PII_CONFIG` (if set)
2. `$HOME/.pii/pii.toml` (falls back to `/root/.pii/pii.toml` if `$HOME` is unset)

The config uses `deny_unknown_fields` throughout: a typo'd key is a load error, never a silently-ignored no-op.

3.2 The `[core]` section

Persistent settings that every entrypoint needs before it can do anything else:

```
[core]
model = "ollama:granite4.2:3b"   # default model spec
runtime_dir = "/root/.pii"       # holds control.sock, graphs/
ollama_host = "http://127.0.0.1:11434"
```

Every entrypoint (`pii -p`, `pii daemon`, `pii tui`, and the runtime-dir lookup) resolves settings in the same order: **CLI flag > environment variable > `[core]` in the config file > built-in default**. Environment variables still win over the config file, so nothing that already relies on env vars changes behavior.

3.3 The `[net]` section

Egress policy knobs:

```
[net]
grants = ["api.example.com:443"]   # host:port patterns every principal may reach
allow_private = false              # allow loopback/private/link-local classes
```

3.4 The `[[worker]]` section

Supervised sub-process workers:

```
[[worker]]
name = "w1"                 # worker name (grant/rpc key)
program = "/bin/pii"        # program to spawn
args = ["worker-exec"]      # argv after the program
env = [["K","V"]]           # extra child environment
max_in_flight = 16          # concurrent in-flight cap (0 = default)
```

3.5 The [[channel]] section

Inbound/outbound channels:

```
[[channel]]
name = "discord1"
kind = "discord"            # discord | cli
graph = "starter/assistant" # graph to run per inbound message
poll_interval_ms = 500      # poll interval (default 500)
# cli kind only:
inbox = "/tmp/in.jsonl"     # inbox file path
outbox = "/tmp/out.jsonl"   # outbox file path
```

3.6 The [memory] section

```
[memory]
path = "mem.redb"           # memory store path (empty = <runtime_dir>/memory.redb)
embedder = "trigram"        # embedder name (only "trigram" ships this phase)
cache_bytes = 0             # cache budget in bytes (reserved)
```

3.7 The [[cron]] section

Scheduled graph runs:

```
[[cron]]
name = "consolidate"        # cron entry name
graph = "starter/consolidate" # graph to run (entry must be a timer node)
schedule = "*/15 * * * *"   # display-only mirror for cron.list
input = {}                  # extra run input merged under "input"
```

3.8 The [evolve] section

Model routing for the self-improving boss (spec 28):

```
[evolve]
routing = { patch = "ollama:granite4.2:3b", localize = "ollama:granite4.2:3b" }
```

The known routing steps are `localize`, `patch`, `review`, `research`, `orchestrator`. An unknown step name is a load error (`deny_unknown_fields` semantics).

3.9 Environment variables

The complete environment surface for this phase:

Variable	Purpose
`PII_CONFIG`	path to the config file
`PII_RUNTIME_DIR`	runtime dir (control.sock, graphs/)
`PII_MODEL`	default model spec
`PII_MEMORY_PATH`	memory store path
`PII_DISCORD_TOKEN`	Discord token (channels)
`PII_DISCORD_CHANNEL_ID`	Discord channel
`PII_DISCORD_API_BASE`	Discord API base
`OLLAMA_HOST`	Ollama host
`OLLAMA_API_KEY`	Ollama API key
`PII_DAEMON_WORKER_THREADS`	daemon worker thread count
`PII_DAEMON_MAX_BLOCKING`	max blocking threads

Variable	Purpose
`PII_TEST_CLOCK_MS`	test-only: fixed clock

3.10 Runtime directory resolution

The runtime directory (which holds `control.sock` and `graphs/`) is resolved in this order:

1. `$PII_RUNTIME_DIR`
2. `[core].runtime_dir` in the config file
3. `$XDG_RUNTIME_DIR/pii/<instance>`
4. `/run/user/<uid>/pii/<instance>`

Running Pii 4

4.1 One-shot prompt (`pii -p`)

The simplest use: run a single prompt and exit.

One-shot

```
pii -p "explain this codebase"
pii -p "fix the bug in src/main.rs" --model ollama:granite4.2:3b
pii -p "summarize" --json          # machine-readable output
```

Flags:

Flag	Purpose
`-json`	emit JSON output
`-model M`	override the model spec
`-graph G`	run a graph instead of the default agent
`-workspace D`	set the workspace directory
`-sandbox off|workspace`	sandbox profile
`-connect auto|loopback|<socket>`	connect mode
`-input J`	JSON input for a graph run

4.2 The daemon (`pii daemon`)

The daemon owns the store and the control socket. Run it in the foreground (or under a process supervisor):

Daemon

```
pii daemon
pii daemon --foreground
pii daemon --config /etc/pii/pii.toml
pii daemon --workspace /srv/pii --store /srv/pii/store.redb
pii daemon --sandbox workspace
```

Flags:

Flag	Purpose
`-config P`	config file path
`-sandbox off|workspace`	sandbox profile
`-workspace D`	workspace directory
`-store S`	store (redb) path
`-foreground`	stay in the foreground

4.3 The TUI (`pii tui`)

A live terminal UI that streams run events:

TUI

```
pii tui
pii tui --connect auto
pii tui --connect loopback
pii tui --connect /run/user/1000/pii/default/control.sock
pii tui --workspace /srv/pii
```

The TUI is a pure protocol client: it holds no domain state, only what the daemon pushes. It shows a header (connection status + session id), a body (the event log, styled by role: user input in green, model output in cyan, progress in dim gray, errors in red), and a footer (the input box). Slash commands: `/exit`, `/quit`, `/help`, `/clear`. While a run is active, the last line shows the live progress indicator `"<prompt> [seq 0x...]"` — the same `seq` number `pii graph trace` prints, so a stuck run is directly cross-referenceable.

4.4 Graphs

Pii's automation engine runs *graphs* — typed assemblies of nodes and edges. The graph engine is documented in the Technical Manual, Chapter 3, and in the SSOT at `_docs/06-research/006-pii-graph-engine.md`.

Graph commands

```
pii graph lint graphs/starter/assistant.toml
pii graph show graphs/starter/assistant.toml
pii graph run graphs/starter/assistant.toml --db /srv/pii/store.redb
pii graph resume <run_id> --db /srv/pii/store.redb
pii graph trace <run_id> --db /srv/pii/store.redb
```

4.5 Control commands

Thin protocol-calling subcommands that connect to the daemon's control socket, call one method, print the JSON result, and exit:

Control

```
pii sys status
pii sys reload
pii run ls
pii run ls --status running
pii run start starter/assistant --input '{"k":"v"}'
pii net ls
pii worker ls
pii worker reload <name>
pii memory search "query" --k 5
pii memory put "text" --layer L1 --kind Atom
pii memory consolidate
pii approve ls
pii approve allow <id> --note "looks good"
pii approve deny <id>
pii heal ls
pii heal show <signature_hash>
pii cron ls
```

4.6 Soak (stress) testing

Soak

```
pii soak --connect /run/user/1000/pii/default/control.sock --runs 3000 --graph
      starter/soak
pii soak --connect <socket> --runs N --graph G --concurrency C
```

5 The Web Viewer

`pii-web` is a read-only web viewer that proxies the daemon's UDS JSON-RPC over loopback HTTP so a browser can watch runs, sessions, approvals, heals, and memory. It is *viewer-only*: no write RPC is proxied.

Web viewer

```
pii-web --port 4602 --host 127.0.0.1 --instance default
# open http://127.0.0.1:4602
```

The daemon must be running for the viewer to proxy to it.

The MCP Bridge 6

`pii-mcp` is an MCP (Model Context Protocol) server + client. The server exposes pii's native tools (fs, edit, bash, glob, grep, http, memory) as an MCP server over stdio, so an MCP client (e.g. Hermes) can call them and exercise pii's engine. The client lets pii reach out and consume external MCP tool servers.

The design is documented in the SSOT at `_docs/02-guides/pii-mcp-bridge.md`.

MCP

```
pii-mcp                    # MCP server over stdio
pii-mcp client <command> [args...]   # spawn an external MCP server, list tools,
      call the first
```

Important

MCP is the *integration surface*, not the control plane. Full control (start a run, watch a trace, resolve an approval) stays on pii's own UDS JSON-RPC. The MCP server exposes pii's *tools*, not the full control API.

7 The Self-Improving Boss

The self-improving boss (`pii-evolve`) is a feature-gated crate that runs the boss loop as a real pii graph on the daemon. The launcher binary is `pii-evolve`:

Boss launcher

```
pii-evolve <state-dir> [--socket PATH]
# --socket overrides the daemon's UDS control socket
# default: $PII_RUNTIME_DIR/control.sock, else ~/.pii/runtime/control.sock
```

Each cycle loads `state.md`, bumps `cycle_count`, sends `run.start` for the `starter/boss-watch` graph, records the launched run id + `phase = "watching"` back into `state.md`, and returns. The daemon owns detection, gating, and the approval ledger.

The deep design is in the SSOT at `_docs/06-research/008-pii-evolve.md`.

8 Debugging

8.1 Check the daemon is up

Status

```
pii sys status
pii --version
```

If `pii sys status` fails in a plain shell, the most common cause is that the daemon was hand-launched with environment variables in a *different* shell. The `[core]` section in the config file backs `model` and `runtime_dir` so settings survive a fresh shell or a daemon restart. See Chapter 3.

8.2 No model configured

Error

```
error: no model configured (set PII_MODEL or --model)
```

Set `PII_MODEL`, pass `-model`, or set `[core].model` in the config file.

8.3 No usable runtime dir

Error

```
error: no usable runtime dir (set PII_RUNTIME_DIR)
```

Set `PII_RUNTIME_DIR` or `[core].runtime_dir`. See Chapter 3 for the resolution order.

8.4 Capability denied

A node fails with `capability denied` when a capability is missing at any of the three layers (run, graph, node). This is a real bug class: a live `ollama:` provider failed because `NET` was missing at the run layer AND at the graph/node layers. See the Technical Manual, Chapter 3, for the capability model.

8.5 Graph lint errors

Lint

```
pii graph lint graphs/starter/assistant.toml
# prints "OK" or a list of rule-coded errors
```

Static lint rejects unknown node kinds, dangling edges, unsatisfiable joins, and cyclic specs without a `loop_budget`. Every error message carries its rule code.

8.6 Tracing a run

Trace

```
pii graph trace <run_id> --db /srv/pii/store.redb
pii graph trace <run_id> --db /srv/pii/store.redb --terminal
pii graph trace <run_id> --db /srv/pii/store.redb --node-status <node>
pii graph trace <run_id> --db /srv/pii/store.redb --count-finished <node>
pii graph trace <run_id> --db /srv/pii/store.redb --max-batch-width
pii graph trace <run_id> --db /srv/pii/store.redb --output
pii graph trace <run_id> --db /srv/pii/store.redb --overlapped <a>,<b>
```

The `seq` counter is the single source of run ordering — the same number the TUI shows as the live `[seq 0x...]` indicator. A live session can be cross-referenced against a persisted trace by seq.

8.7 Approvals

When a graph suspends for human approval (e.g. a heal-fix gate), resolve it with `pii approve`:

Approve

```
pii approve ls
pii approve allow <id> --note "approved"
pii approve deny <id>
```

8.8 Heal ledger

The self-healing ledger records attempts. Inspect it with `pii heal`:

Heal

```
pii heal ls
pii heal show <signature_hash>
```

8.9 Logs

The daemon writes to stderr. Run it in the foreground and capture stderr, or run it under a process supervisor that captures logs. There is no separate log file by default.

9 Operational Notes

9.1 Single-writer store

The daemon owns the redb store (single-writer MVCC ACID). Only one daemon should write to a given store at a time. The control socket is local-only Unix (mode 0600), and peer-credential authorization rejects a peer whose uid differs from the daemon's.

9.2 Feature-gated subsystems

The default build is a monolith minus gated subsystems. Network/token subsystems (channels, TLS, github) are opt-in features — a feature that *excludes the code* is stronger than a runtime config that *disables* it. See `_docs/02-guides/feature-gating.md`.

9.3 Where to go for more

- **Architecture**: the companion Technical Manual.
- **Research SSOT**: `_docs/06-research/` (numbered research write-ups).
- **Guides**: `_docs/02-guides/`.
- **Vision**: `_docs/01-overview/001-pii-vision.md`.